

CME 296 – Lecture 1: Diffusion — poznámky ke studiu

Zdroje: [oficiální slidy k lekci 1](#) + [web kurzu](#). Kurz **CME 296 – Diffusion & Large Vision Models** (Stanford) vedou Afshine a Shervine Amidi — stejná dvojice, co stojí za CME 295 (Transformers & LLMs) a za sérii "Super Study Guide" cheatsheetů.

Shrnutí v kostce

Difuzní model = dva procesy: **forward** (známý, přidává šum) a **reverse** (učený, odšumuje)

Trénink: síť se učí **predikovat přidaný šum** — to je finální trik po celém ELBO odvození

Generování: iterativní odšumování z čistého šumu, typicky ~1000 kroků → **pomalé**

DDIM: stejná natrénovaná síť, jiný (deterministický) generační proces → **10-100× rychlejší**, za cenu kvality

Obsah

Kam lekce zapadá do kurzu

Motivace: proč generovat obrázky ze šumu

Dva klíčové procesy: forward a reverse

Jak reprezentujeme obrázek a šum

Forward proces – matematika

Cíl: maximum likelihood

Odvození loss funkce (ELBO) – jádro DDPM

Trénovací algoritmus

Generování (inference) – algoritmus

Problém: DDPM je pomalý

Řešení: DDIM

Propojení s praxí a výzkumným směrem

Glosář

Doplňkové zdroje

0. Kam lekce 1 zapadá do kurzu

Kurz stojí na 4 pilířích: **generation paradigm** (jak model generuje), **model architecture** (čím generuje), **model training** (jak se to natrénuje) a **evaluation** (jak poznáme, že je to dobré).

L1-L3 = generation paradigms. Podle popisu kurzu ("foundations of diffusion, score matching and flow matching") to nejspíš znamená: dnešní diffusion (L1), score-based/SDE pohled (pravděpodobně L2) a flow matching (pravděpodobně L3) — tři různá "paradigmata", jak obrázky generovat.

L4 a dál = architektury (U-Net, Diffusion Transformer), trénování a evaluace — přesné rozdělení lekcí ze slidů jasné není, ale to je zhruba zbytek obsahu kurzu podle webu.

Lekce 1 je čistě o **unconditioned** generování (bez textového promptu) — conditioning (text→image) přijde až později. Předpokládané znalosti: lineární algebra, pravděpodobnost (Bayes, marginály, Gaussovo rozdělení), diferenciální rovnice (ODE/SDE) a základy ML.

1. Motivace: proč generovat obrázky ze šumu?

Cíl: model, který umí vygenerovat nový "věrohodný" obrázek vypadající, jako by pocházel ze stejného rozdělení jako trénovací data — ale není kopií žádného konkrétního tréninkového vzorku.

Proč startovat od čistého šumu?

šum se triviálně vzorkuje (na rozdíl od "vzorkování obrázku")

vnáší nutnou náhodnost — jinak by model pro stejný vstup pořád generoval to samé

Gaussovo rozdělení má matematicky pohodlné vlastnosti (uzavřené vzorce pro součty, marginály, podmíněná rozdělení), což se ukáže jako klíčové v odvození níže

"The sculpture is already complete within the marble block, before I start my work. It is already there, I just have to chisel away the superfluous material." — Michelangelo (citováno na slidech jako analogie)

Obrázek je "schovaný" uvnitř šumu, model se jen učí "odsekávat" šum, dokud se obrázek neodhalí — podobně jako sochař neodhaluje sochu náraz, ale kousek po kousku.

2. Dva klíčové procesy

Forward proces

Co dělá čistý obrázek $\rightarrow$ čím dál šumnější šum $\rightarrow$ čím dál čistší obrázek

Známe ho? **ano** — definujeme si ho sami **ne** — síť se ho musí **naučit**

Náhodnost ano, přidáváme náhodný šum v DDPM ano; u DDIM (viz níže) lze odstranit

Celý trik difuze: "kažení" dat (přidávání šumu) je triviální a známe ho analyticky. Když se navíc naučíme udělat jeden krok zpátky (odšumit o kousek), stačí to zopakovat mnohokrát a dostaneme se z čistého šumu zpátky k obrázku.

Reverse proces

3. Jak reprezentujeme obrázek a šum

Obrázek = mřížka šířka $\times$ výška, každý pixel = (R, G, B) $\in [0,255]^3$. Matematicky je to jeden (velmi dlouhý) vektor v $\mathbb{R}^d$.

Rozdělení pravděpodobnosti:

v 1D: skalár popsán střední hodnotou (expectation) a rozptylem (variance)

ve vysoké dimenzi: vektor popsán střední hodnotou (vektorem) a **kovarianční maticí**

Klíčový zjednodušující předpoklad celého kurzu: skoro pořád se pracuje jen s **izotropními Gaussovkami** — kovarianční matice = $\sigma^2 \cdot I$ (stejný rozptyl ve všech směrech). Díky tomu má hustota pravděpodobnosti jednoduchý uzavřený tvar, se kterým se dá reálně počítat.

Šum je i.i.d. (independent and identically distributed).

Noise schedule (β_t) — posloupnost čísel (rostoucí, blízko 0 na začátku, blízko 1 na konci), určující kolik šumu se přidá v kroku t . Je to hyperparametr volený předem.

4. Forward proces – matematika

Krok po kroku (Markovův řetězec):

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \cdot \mathbf{x}_{t-1} + \sqrt{\beta_t} \cdot \boldsymbol{\epsilon}_t, \text{ kde } \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Protože je to řetězec samých Gaussovek a *rozptyl součtu nezávislých Gaussovek = součet rozptylů*, dá se odvodit uzavřený vzorec skákající přímo z $\mathbf{x}_0$ do libovolného $\mathbf{x}_t$ (bez procházení mezikroků). Označme $\alpha_t := 1 - \beta_t$ a $\bar{\alpha}_t := \alpha_1 \cdot \alpha_2 \cdot \dots \cdot \alpha_t$:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}, \text{ kde } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Tohle je jeden z nejdůležitějších vzorců v celém oboru — používá se neustále, v tréninku i v inferenci.

Konkrétní příklad na číslech (pro intuici; $\beta_1 = 0.1$, $\beta_2 = 0.2$, $\mathbf{x}_0 = 1$):

$$\alpha_1 = 0.9 \rightarrow \bar{\alpha}_1 = 0.9 \rightarrow x_1 = 0.949 \cdot x_0 + 0.316 \cdot \epsilon$$

$$\alpha_2 = 0.8 \rightarrow \bar{\alpha}_2 = 0.9 \times 0.8 = 0.72 \rightarrow x_2 = 0.849 \cdot x_0 + 0.529 \cdot \epsilon$$

S rostoucím t klesá koeficient u "signálu" ($\mathbf{x}_0$) a roste koeficient u šumu — po T krocích (s dobře zvoleným schedulem) je $\bar{\alpha}_T \approx 0$, takže $\mathbf{x}_T \approx$ čistý šum $\mathcal{N}(\mathbf{0}, \mathbf{I})$, nezávisle na původním obrázku.

5. Cíl: maximum likelihood

Hledáme parametry θ (váhy sítě) maximalizující **(log-)věrohodnost** trénovacích obrázků:

$$\max_{\theta} \sum_{\mathbf{x}_0 \in \text{data}} \log p_{\theta}(\mathbf{x}_0)$$

(log kvůli numerické stabilitě — mění součiny na součty, nemění polohu maxima)

Problém: $p_{\theta}(\mathbf{x}_0) = \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$ vyžaduje integrovat přes všechny možné "cesty" šumu, což není přímo spočitatelné. Řešení: **variational lower bound** (další sekce). Slidy si tu ještě krátce zopakují aparát potřebný pro odvození — sdruženou (joint) pravděpodobnost, marginalizaci, podmíněnou pravděpodobnost a Bayesovo pravidlo.

6. Odvození loss funkce (ELBO) — jádro DDPM

Strategie o čtyřech krocích:

1) Odvod' dolní mez. Jensenova nerovnost aplikovaná na vhodné variační rozdělení $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ dá **ELBO (Evidence Lower Bound)**:

$$\log p_{\theta}(\mathbf{x}_0) \geq \mathbb{E}_q[\log p_{\theta}(\mathbf{x}_{0:T}) - \log q(\mathbf{x}_{1:T}|\mathbf{x}_0)] =: \text{ELBO}$$

2) Rozepiš členy. Vlastnosti logaritmu + přeuspořádání dají ELBO jako součet tří typů členů (standardní rozklad z DDPM paperu):

L_T — "prior matching": KL divergence mezi $q(x_T|x_0)$ a $p(x_T)=N(0,I)$. Při dobrém schedulu skoro nulová/konstantní $\rightarrow$ dá se ignorovat.

L_{t-1} pro $t = 2, \dots, T$ — **hlavní členy**: KL divergence mezi $q(x_{t-1}|x_t, x_0)$ (co bychom "měli" udělat, kdybychom znali x_0) a $p_\theta(x_{t-1}|x_t)$ (co model reálně dělá)

L_0 — rekonstrukční člen pro poslední krok

3) Ukaž tractabilitu:

(a) $q(x_{t-1}|x_t, x_0)$ lze spočítat v uzavřeném tvaru přes **Bayesovo pravidlo + Markovovu vlastnost** — je to opět Gaussovka (celý řetězec je Gaussovský)

(b) $p_\theta(x_{t-1}|x_t)$ se **předpokládá** Gaussovské — odůvodnění: při dostatečně malých krocích je reverse proces (přibližně) taky Gaussovský

4) Odvod' finální loss. KL divergence dvou Gaussovek má uzavřený tvar (závisí na rozdílu středních hodnot váženém rozptylem). Když se střední hodnota modelu p_θ parametrizuje tak, aby predikovala **šum ϵ** místo přímo x_{t-1} nebo x_0 , celý výraz se zjednoduší na překvapivě jednoduchou věc:

$$L_{\text{simple}}(\theta) = E_{\{t, x_0, \epsilon\}} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

Síť ϵ_θ se tedy **jen učí predikovat, jaký šum byl do obrázku přidán** — to je celé tajemství DDPM tréninku. Kompletní odvození se všemi algebraickými kroky je v [Ho et al., 2020](#) — slidy ho označují jako "landmark paper, must-read".

7. Trénovací algoritmus

Vezmi trénovací obrázek x_0 z datasetu

Vzorkuj časový krok $t \sim \text{Uniform}(1, \dots, T)$

Vzorkuj šum $\epsilon \sim N(0, I)$

Spočítej zašuměný obrázek: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$

Nech síť predikovat šum: $\epsilon_\theta(x_t, t)$

Loss = $\|\epsilon - \epsilon_\theta(x_t, t)\|^2 \rightarrow$ gradient step (backprop)

Opakuj

Síť **nikdy neběží iterativně** při trénování — pro každý krok se náhodně vybere jedno t a síť se natrénuje na predikci šumu jen pro tenhle jeden krok. Iterativnost přichází až u generování.

8. Generování (inference) — algoritmus

Vzorkuj $x_T \sim N(0, I)$ (čistý šum)

Pro $t = T, T-1, \dots, 1$ opakuj: síť predikuje šum $\epsilon_\theta(x_t, t)$, který se z x_t odečte; přidej trochu nového náhodného šumu (kromě posledního kroku) $\rightarrow x_{t-1}$

Vrať x_0 — vygenerovaný obrázek

Tomuto typu vzorkování se říká **ancestral sampling** — je to přesně obrácený proces k forward difuzi, prováděný krok po kroku.

9. Problém: DDPM je pomalý

Generování vyžaduje **T sekvenčních kroků** (v originále typicky $T = 1000$)

Řádově dražší než GAN/VAE (jeden forward pass) → generování jednoho vzorku může trvat **řádově minuty**

→ "V praxi je DDPM při inferenci příliš pomalý!"

10. Řešení: DDIM (rychlejší sampling)

Neúspěšný pokus 1: odvodit vzorec přímo z x_t na x_{t-k} induktivním rozepsáním rekurence. Problém: síť ϵ_θ se pořád musí vyhodnotit stejně mnohokrát — **žádný reálný speedup**.

Neúspěšný pokus 2 ([Song et al., 2020](#) — DDIM paper): co kdybychom při generování prostě přeskakovali kroky (síť natrénovaná na T kroků, vzorkování jen na podmnožině časů)? Problém: **velké skoky + ponechaná náhodnost mezi kroky = výrazně horší kvalita**.

Přeformulování problému: najdi jiný forward proces (nemusí být Markovovský), který:

má **stejné marginály** $q(x_t|x_0)$ jako DDPM — klíčové pozorování: loss funkce z kroku 6 závisí **jen na těchto marginálech**, ne na celém řetězci, takže síť natrénovaná pro DDPM jde použít 1:1, **bez přetrénování** jehož generační proces **lze udělat deterministickým**

Klíčový trik: z predikovaného šumu $\epsilon_\theta(x_t, t)$ lze odhadnout, jak by vypadal čistý obrázek ("predikce $\hat{x}_0$ "):

$$\hat{x}_0(x_t, t) = (x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$$

Tuhle predikci pak **deterministicky "přešumíme" zpátky** na menší časový krok, stejným vzorcem jako ve forward procesu, ale s ϵ_θ místo skutečného šumu a **bez přidání nové náhodnosti**:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{x}_0(x_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t)$$

To je (deterministická) **DDIM update rule**. (Obecná DDIM formulace má navíc parametr η , který plynule interpoluje mezi tímto deterministickým chováním ($\eta=0$) a DDPM-like stochastickým chováním ($\eta=1$) — na slidech není explicitně, ale je dobré o něm vědět.) Protože je krok deterministický, nic nebrání přeskočit rovnou z t na $t-k$ (třeba o 20 kroků) — "výtah" mezi dvěma libovolnými časy funguje stejně dobře jako mezi sousedními.

Proč "Implicit"? DDIM = Denoising Diffusion **Implicit** Models. Vzorky vznikají pevným (deterministickým) postupem z latentní proměnné x_T , čemu se říká *implicitní pravděpodobnostní model* — ale síť je natrénovaná úplně stejně (stejnou loss funkcí) jako u DDPM.

Rychlost vs. kvalita (CIFAR-10, metrika FID — čím nižší, tím lepší):

Zrychlení 1× (baseline) 10× 20× 50× 100×

Změna FID — +3 % +16 % +70 % +330 %

Typický speedup je **10–100×**, vždy ale jde o kompromis — čím agresivněji se přeskakuje, tím rychleji se kvalita zhoršuje. Volba konkrétních kroků k přeskočení (schedule) je hyperparametr.

DDIM recept (shrnutí):

Zvol modelovací předpoklady kompatibilní s DDPM optimalizací (stejně marginály)

Odstraň mezikrokovou stochasticitu → DDIM

Přeskakuj kroky!

11. Propojení s praxí a výzkumným směrem

DDIM je historicky **první** a koncepčně nejjednodušší člen z celé rodiny "training-free" rychlých samplerů — přesně to, co se dnes v nástrojích jako Stable Diffusion / ComfyUI schovává za volbou "sampleru" (DDIM, PNDM, Euler, DPM++ a další jsou různé strategie, jak odhadnout tutéž trajektorii v co nejmeně krocích).

Přesah do SDE: DDIM update rule je ve skutečnosti diskretizace tzv. **probability flow ODE** — té samé rovnice, na které stojí score-based/SDE formulace difuze. I když se to tu odvozuje čistě přes DDPM, koncepčně je to první krok směrem ke kontinuálnímu pohledu na difuzi, který zřejmě naplno rozvedou lekce 2–3.

Pro směr okolo **adaptivních/rychlejších samplerů** jako možného výzkumného tématu je tohle přesně fundament: DDIM (pevný počet přeskočených kroků) je baseline, na kterou později navazují např. DPM-Solver nebo samplery, co si počet kroků volí adaptivně.

12. Glossář

Forward proces — dopředné, řízené přidávání šumu; známe ho analyticky

Reverse proces — zpětné odšumování; toto se síť učí

β_t (noise schedule) — kolik šumu přidáme v kroku t

$\alpha_t, \bar{\alpha}_t$ — $1 - \beta_t$, resp. kumulativní součin α do času t

ELBO — Evidence Lower BOund, dolní mez na log-věrohodnost, kterou reálně optimalizujeme

$\epsilon_\theta(\mathbf{x}_t, t)$ — neuronová síť predikující přidáný šum

DDPM — Denoising Diffusion Probabilistic Models (Ho et al., 2020)

DDIM — Denoising Diffusion Implicit Models (Song et al., 2020)

Izotropní Gaussovka — Gaussovo rozdělení se stejným rozptylem ve všech směrech (kovariance = $\sigma^2 \cdot \mathbf{I}$)

FID (Fréchet Inception Distance) — metrika kvality generovaných obrázků; nižší = lepší

13. Doplnkové zdroje

[Sylabus kurzu](#)

[Veřejný cheatsheet kurzu](#)

[VIP Cheatsheet \(PDF na GitHubu\)](#)

Ho, Jain, Abbeel (2020): *Denoising Diffusion Probabilistic Models* — [arXiv:2006.11239](#) · [oficiální kód](#)

Song, Meng, Ermon (2020): *Denoising Diffusion Implicit Models* — [arXiv:2010.02502](#) · [oficiální kód](#)

Sesterský kurz: [CME 295 – Transformers & Large Language Models](#) (stejní vyučující)